Practical 5 – Generate Binomial Coefficients Using Dynamic Programming

1. Understanding the problem statement

Problem statement: "Write a program to generate binomial coefficients using Dynamic Programming."

Let's break it down:

- Binomial coefficient: A mathematical value that tells us the number of ways to select a particular number of items from a given number of items.
- Generate: Calculate the required binomial coefficient.
- Dynamic Programming: Solve the problem by breaking it into smaller subproblems and storing their results to avoid repeated calculations.

In simple words, we need to write a Java program that calculates the binomial coefficient using Dynamic Programming.

2. What is a Binomial Coefficient?

A binomial coefficient represents the number of ways to select `r` objects from `n` objects, without considering the order of selection.

It is represented as:

$$\binom{n}{r}$$

It is also written as:

$$C(n, r)$$

Example:

Suppose you have 5 students and you need to select 2 students for a competition.

How many different pairs can you form?

The answer is:

$$C(5, 2) = 10$$

This means we can form 10 different pairs.

Formula of Binomial Coefficient

The mathematical formula is:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

Here:

- n = Total number of objects.
- r = Number of objects we want to select.
- ! = Factorial.

What is factorial?

Factorial means multiplying a positive integer by all the positive integers below it until 1.

For example:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Similarly:

$$3! = 3 \times 2 \times 1 = 6$$

Let's calculate our example:

$$C(5, 2) = \frac{5!}{2!(5-2)!}$$

$$= \frac{5\times4\times3\times2\times1}{(2\times1)(3\times2\times1)}$$

\[=\frac{120}{12}=\boxed{10} \]

So, the answer is 10.

Where are Binomial Coefficients used?

- Probability and statistics.
- Combinatorics (counting possibilities).
- Pascal's Triangle.
- Algebra, especially binomial expansion.

- Computer science algorithms.

3. What is Pascal's Triangle?

Before understanding the Dynamic Programming approach, we need to understand Pascal's Triangle.

Pascal's Triangle is a triangular arrangement of numbers in which:

1. Every row starts with 1.
2. Every row ends with 1.
3. Every middle number is the sum of the two numbers directly above it.

Let's write it:

```
None
                1
              1   1
            1   2   1
          1   3   3   1
        1   4   6   4   1
      1   5  10  10   5   1
```

Each row represents a value of n.

For example:

n	Binomial coefficients
0	1
1	1 1
2	1 2 1
3	1 3 3 1

4 1 4 6 4 1

5 1 5 10 10 5 1

Look at row 5:

```
None
1  5  10  10  5  1
```

The coefficient at position 2 is 10.

Therefore:

$$C(5, 2) = 10$$

Important: The first position in each row is position 0, not position 1.

4. How do we generate Binomial Coefficients using Dynamic Programming?

Now comes the most important part.

We can calculate binomial coefficients using this recurrence relation:

$$C(n, r) = C(n - 1, r - 1) + C(n - 1, r)$$

This means:

To calculate a middle value, add the two values directly above it in Pascal's Triangle.

For example:

$$C(5, 2) = C(4, 1) + C(4, 2)$$

From Pascal's Triangle:

$$= 4 + 6 = 10$$

So, we get 10 without directly calculating factorials.

Two important conditions (Base Cases)

There are two situations where the answer is always 1.

Condition 1: When r = 0

$$C(n, 0) = 1$$

There is exactly one way to select zero objects: select nothing.

Condition 2: When r = n

$$C(n, n) = 1$$

There is exactly one way to select all n objects: select every object.

These are called base cases.

Why use Dynamic Programming?

Suppose we want to calculate:

$$C(5, 2)$$

Using the recurrence:

$$C(5, 2) = C(4, 1) + C(4, 2)$$

And:

$$C(4, 2) = C(3, 1) + C(3, 2)$$

Several smaller calculations are needed. Some subproblems can occur repeatedly.

Instead of calculating them again and again, Dynamic Programming stores the answers in a table and reuses them.

This is why DP is useful here.

5. Which Dynamic Programming approach have we used?

We have used Tabulation (Bottom-Up Dynamic Programming), just like our previous Knapsack practical.

Remember:

- Memoization = Top-Down = Recursion + Storage.
- Tabulation = Bottom-Up = Iteration + DP Table.

Our program uses nested `for` loops to fill the table from smaller values to larger values.

How does the table work?

We create a two-dimensional array:

```
None
int[][] dp = new int[n + 1][r + 1];
```

Each cell stores a binomial coefficient.

`dp[i][j]` represents:

$$C(i, j)$$

For example:

`dp[5][2]` represents $C(5, 2)$, which is 10.

6. Complete code explanation

Let's understand your exact code now.

Part 1: Import Scanner

```
None
import java.util.Scanner;
```

Imports the Scanner class to accept input from the user.

Part 2: Class and main method

```
None
public class binomial {

    public static void main(String[] args) {
```

- `public class binomial`: Defines our class.
- `main()`: Starting point of Java program execution.
- `static`: Allows Java to execute main without creating an object.
- `void`: Means the method does not return a value.

Part 3: Create Scanner object

```
None
Scanner sc = new Scanner(System.in);
```

Creates a Scanner object to read input from the keyboard.

Part 4: Accept n and r

```
None
System.out.print("Enter value of n: ");
int n = sc.nextInt();

System.out.print("Enter value of r: ");
int r = sc.nextInt();
```

We accept two values:

- `n`: Total number of objects.
- `r`: Number of objects to select.

For example:

```
None
n = 5
r = 2
```

We want to calculate $C(5, 2)$.

Part 5: Input validation

```
None
if (n < 0 || r < 0 || r > n) {
    System.out.println("Invalid input");
    sc.close();
    return;
```

```
}
```

This checks whether the input is valid.

- `n < 0`: Total objects cannot be negative.
- `r < 0`: Number of selected objects cannot be negative.
- `r > n`: We cannot select more objects than the total available.

`||` means OR.

If any one of these conditions is true, the input is invalid.

`return` stops the execution of the main method.

Part 6: Create the DP table

```
None
int[][] dp = new int[n + 1][r + 1];
```

This creates a two-dimensional array to store the calculated binomial coefficients.

For our example:

```
None
int[][] dp = new int[6][3];
```

Why 6 rows and 3 columns?

- `n + 1` = 6 rows.
- `r + 1` = 3 columns.

We need the extra row and column to represent zero values.

Part 7: Outer loop

```
None
for (int i = 0; i <= n; i++) {
```

This loop processes values from 0 to n.

For our example, it runs from:

```
None
i = 0, 1, 2, 3, 4, 5
```

Each value represents a row of Pascal's Triangle.

Part 8: Inner loop

```
None
for (int j = 0; j <= Math.min(i, r); j++) {
```

This loop calculates coefficients for each row.

Let's understand `Math.min(i, r)`.

It returns the smaller of the two values.

Why do we need it?

Because a binomial coefficient is valid only when:

$$0 \leq j \leq i$$

Also, we only need columns up to `r`, because the user wants $C(n, r)$.

For example, when `i = 2` and `r = 2`, the loop runs from 0 to 2.

When `i = 1` and `r = 2`, it runs only from 0 to 1.

Part 9: Checking the base cases

```
None
if (j == 0 || j == i) {
    dp[i][j] = 1;
}
```

This checks the two base cases:

- When `j == 0`, the answer is 1.
- When `j == i`, the answer is 1.

For example:

$$C(4, 0) = 1$$

$$C(4,4) = 1$$

These values are directly stored in the DP table.

Part 10: Applying the recurrence relation

```
None
else {
    dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
}
```

This is the main logic of the program.

If the current position is not a boundary position, we add the two values directly above it.

For example:

$$C(5,2) = C(4,1) + C(4,2)$$

$$= 4 + 6 = 10$$

In array form:

```
None
dp[5][2] = dp[4][1] + dp[4][2];
```

The result is stored in `dp[5][2]`.

Part 11: Display the result

```
None
System.out.println("Binomial Coefficient C(" + n + ", " + r + ")
= " + dp[n][r]);
```

This displays the final answer.

Why `dp[n][r]`?

Because we want the coefficient for the given values of `n` and `r`.

For our example:

```
None
dp[5][2] = 10
```

Part 12: Close Scanner

```
None
sc.close();
```

Closes the Scanner object after taking input.

7. Complete dry run

Let's take:

```
None
n = 5
r = 2
```

We want to calculate:

$$C(5, 2)$$

Our DP table has 6 rows and 3 columns.

We fill the table row by row.

Row 0:

Since `j = 0` and `j = i`, the value is 1.

```
None
1
```

Row 1:

Both positions are boundary positions.

```
None
1   1
```

Row 2:

- `C(2,0) = 1`
- `C(2,1) = C(1,0) + C(1,1) = 1 + 1 = 2`
- `C(2,2) = 1`

```
None
1  2  1
```

Row 3:

- `C(3,0) = 1`
- `C(3,1) = 1 + 2 = 3`
- `C(3,2) = 2 + 1 = 3`

```
None
1  3  3
```

Row 4:

- `C(4,0) = 1`
- `C(4,1) = 1 + 3 = 4`
- `C(4,2) = 3 + 3 = 6`

```
None
1  4  6
```

Row 5:

- `C(5,0) = 1`
- `C(5,1) = 1 + 4 = 5`
- `C(5,2) = 4 + 6 = 10`

```
None
1  5  10
```

Final DP table

Values stored in our DP table

Row (i)	j = 0	j = 1	j = 2

0	1	–	–
1	1	1	–
2	1	2	1
3	1	3	3
4	1	4	6
5	1	5	10

The dash indicates a cell that our program does not need to calculate.

Final answer: C(5, 2)

10

8. Time and space complexity

Time Complexity: O(n × r)

Our code contains two nested loops.

```
None
for (int i = 0; i <= n; i++) {
    for (int j = 0; j <= Math.min(i, r); j++) {
```

- Outer loop runs `n + 1` times.
- Inner loop runs at most `r + 1` times.
- Each calculation takes constant time, O(1).

Therefore:

\[\boxed{O(n \times r)} \]

Space Complexity: O(n × r)

We create a two-dimensional array:

```
None
int[][] dp = new int[n + 1][r + 1];
```

It requires space proportional to the number of rows multiplied by the number of columns.

Therefore:

\[\boxed{O(n \times r)} \]

9. Advantages and disadvantages

Advantages:

1. Avoids repeated calculations by storing previous results.
2. Uses a simple recurrence relation.
3. Does not require recursion.
4. Efficiently generates binomial coefficients for moderate values of n and r.

Disadvantages:

1. Requires additional memory for the DP table.
2. Large binomial coefficients may exceed the range of Java's `int` data type.
3. If only one coefficient is needed, the entire table may not always be necessary.

10. Important viva questions

Q1. What is a binomial coefficient?

It represents the number of ways to select `r` objects from `n` objects without considering order.

Q2. What is its notation?

It is represented as $C(n, r)$ or $\binom{n}{r}$.

Q3. What is the formula for a binomial coefficient?

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

Q4. What is factorial?

Factorial is the product of all positive integers from 1 to the given number. For example, 5! = 120.

Q5. What is Pascal's Triangle?

It is a triangular arrangement of numbers where each middle number is the sum of the two numbers directly above it.

Q6. What is the recurrence relation?

$$C(n, r) = C(n - 1, r - 1) + C(n - 1, r)$$

Q7. What are the base cases?

$$C(n, 0) = 1$$

$$C(n, n) = 1$$

Q8. Which Dynamic Programming approach have you used?

Tabulation, also called Bottom-Up Dynamic Programming.

Q9. Why do we use Dynamic Programming?

To store previously calculated values and avoid repeated calculations.

Q10. What does `dp[i][j]` represent?

It represents the binomial coefficient $C(i, j)$.

Q11. Why do we use a two-dimensional array?

To store binomial coefficients for different values of `i` and `j`.

Q12. What does `Math.min(i,r)` do?

It returns the smaller value of `i` and `r`, ensuring that we calculate only the required valid positions.

Q13. What is the time complexity?

O(n × r).

Q14. What is the space complexity?

O(n × r).

Q15. Can we solve this problem using recursion?

Yes, using the recurrence relation, but simple recursion recalculates overlapping subproblems. DP avoids that by storing results.

Q16. What is the difference between Memoization and Tabulation?

Memoization uses a top-down recursive approach, whereas Tabulation uses a bottom-up iterative approach.

Q17. Where are binomial coefficients used?

In probability, statistics, combinatorics and binomial expansion.

Q18. Why is C(n,0) always 1?

Because there is exactly one way to select zero objects: select nothing.

Q19. Why is C(n,n) always 1?

Because there is exactly one way to select all the objects: select every object.

Q20. What happens if r is greater than n?

The input is invalid because we cannot select more objects than the total number available.

11. Quick revision

Topic	Answer
Problem	Generate binomial coefficients
Meaning	Number of ways to select r objects from n
Notation	C(n,r)
Formula	n! / (r! × (n-r)!)
Recurrence	C(n-1,r-1) + C(n-1,r)

Base cases	C(n,0) = 1 and C(n,n) = 1
Triangle	Pascal's Triangle
DP technique	Tabulation
Approach	Bottom-Up
Time complexity	O(n × r)
Space complexity	O(n × r)
Example	C(5,2) = 10